

NLP Final Task 2026

RAG-Based E-commerce Customer Support Chatbot

Introduction

This project requires designing and implementing a Retrieval-Augmented Generation (RAG) based chatbot for e-commerce customer support. The chatbot must provide grounded, accurate, and tone-appropriate responses to customer queries about orders, refunds, deliveries, accounts, and other common retail support topics.

The system integrates multiple NLP techniques into a single pipeline, where each component contributes directly to the overall performance of the final system — this is a fully integrated, end-to-end project rather than isolated tasks.

System Overview

The chatbot processes every customer message through four stages before producing a final response:

- Language Detection: identifies the language of the customer's message, so the system can search the knowledge base correctly and reply in the same language.
- Sentiment/Emotion Classification: identifies whether the customer sounds frustrated, neutral, or satisfied — critical for adjusting tone (an angry customer needs a more apologetic, urgent response than a neutral one).
- Intent Classification: routes the message to the correct handling path based on what the customer actually wants.
- Q&A RAG: retrieves grounded information from the customer-support knowledge base to answer the query accurately.

Project Modules

1) Language Detection

Build a multi-class classifier using traditional NLP (Count Vectorizer or TF-IDF Vectorizer) to classify the language of the customer's message. This is important for searching the right knowledge base entries and replying in the same language the customer used. >>> do your enhancements

2) Sentiment / Emotion Classifier

Build a multi-class classifier using either Recurrent Neural Networks or Transformers to classify the emotional tone of the customer's message (e.g. frustrated/negative, neutral, satisfied/positive). This is a key factor in the system's success: a frustrated customer's message should trigger a more empathetic, priority-flagged response path.

3) Intent Classifier

Build a multi-class classifier using either traditional ML on the labeled intent column (recommended, since the dataset already provides gold intent labels), or zero/few-shot LLM prompting, to classify the customer's intent. Recommended intent categories, condensed from the dataset's 27 fine-grained intents into a manageable set for routing:

- greeting / goodbye / gratitude (small talk, no retrieval needed)
- order_status (track_order, delivery_options, delivery_period)
- order_management (cancel_order, change_order, place_order)
- billing_and_refunds (check_invoice, get_refund, payment_issue)
- account_management (create_account, edit_account, delete_account, switch_account, recover_password)
- complaint (complaint, review — should be flagged for priority handling regardless of RAG)
- out_of_scope

This module is important for routing the entire system to the best response path — e.g. a greeting needs no RAG and can be answered directly; an order-status question must use RAG grounded in the knowledge base; a complaint might route to a human-escalation message rather than a generated answer.

4) Q&A RAG

Build a RAG pipeline using the customer-support dataset to answer incoming customer questions. Use any suitable framework, or build from scratch. Follow these components:

- Vector database: free cloud Qdrant (or a local FAISS/Chroma store if you want to avoid external account setup).
- Embeddings: sentence-transformers (e.g. all-MiniLM-L6-v2).
- LLM: free Groq account, using gpt-oss-120b or gpt-oss-20b.

Datasets

1) Language Identification Dataset

- papluca/language-identification — 90k samples, 20 languages, pre-split train/val/test.

<https://huggingface.co/datasets/papluca/language-identification>

2) Sentiment / Emotion Dataset

- dair-ai/emotion — 20k English Twitter messages labeled with 6 emotions (sadness, joy, love, anger, fear, surprise). Map to 3 buckets (negative/neutral/positive) if a simpler tone signal is preferred for routing.

<https://huggingface.co/datasets/dair-ai/emotion>

Note the domain shift: this dataset is Twitter text, not customer-support text. If time allows, consider supplementing with a small hand-labeled set of actual customer-support-style frustrated/neutral/happy messages for a more realistic qualitative check.

3) Customer Support Knowledge Base (Intent + RAG)

- bitext/Bitext-customer-support-llm-chatbot-training-dataset — 26,872 instruction/response pairs across 27 intents and 10 categories (ACCOUNT, ORDER, INVOICE, DELIVERY, FEEDBACK, CANCELLATION_FEE, NEWSLETTER, PAYMENT, REFUND, SHIPPING_ADDRESS).

<https://huggingface.co/datasets/bitext/Bitext-customer-support-llm-chatbot-training-dataset>

This single dataset conveniently serves two purposes:

- Intent classifier training data: the 'intent' column is already gold-labeled — this is a supervised classification task, not zero/few-shot, if you use this column directly (a nice simplification over the mental-health version, which had no labeled intent data available).
- RAG knowledge base: the 'instruction' (customer question) and 'response' (expected agent answer) columns form ready-made chunks — embed the instructions for retrieval, and inject the paired responses into the generation prompt as grounding context.

Bitext also publishes several vertical-specific variants if you want a narrower or different domain flavor, e.g. Bitext-retail-banking-llm-chatbot-training-dataset, Bitext-events-ticketing-llm-chatbot-training-dataset, or others across 20 verticals (see bitext.com/chatbot-verticals for the full list).

Suggested Prompt Template (RAG Generation Step)

```
System: "You are a helpful, professional customer support assistant for an online retailer. Answer the customer's question using ONLY the information in the retrieved support responses below. If the customer sounds frustrated ({detected_sentiment}), acknowledge that before answering. If the retrieved context does not cover the question, say so honestly and offer to escalate to a human agent rather than guessing."
```

```
Context (retrieved past support responses):  
{retrieved_chunk_1}
```

```
{retrieved_chunk_2}  
{retrieved_chunk_3}
```

Customer question: "{user_message}"

Guidelines

- You must use Python.
- Choose the most suitable data pre-processing techniques for each module.
- Use Flask or FastAPI (or any suitable framework) to deploy the model locally.
- Route complaint/negative-sentiment messages distinctly — e.g. prepend an apology/acknowledgment before the RAG-generated answer, or flag for human escalation instead of auto-responding, depending on your design choice (document whichever you pick and why).

Deliverables

- Four module-specific notebooks (language detection, sentiment classifier, intent classifier, RAG pipeline).
- Deployment scripts.
- Any additional files/documentation you need.

Notes

In the assessment phase, you'll be asked to run your models locally, and you'll be asked about any technical decision or implementation choice you've made — so be well prepared, and avoid overcomplicated approaches you don't fully grasp.